ResQAI: AN INTELLIGENT DISASTER RESPONSE CITIZEN APPLICATION

A Mobile-Centred Framework for SOS Initiation, Location-Aware Reporting, and Citizen Support

saisanjay.ss.2024.cse@rajalakshmi.edu.in

[2]Department of Computer Science and Engineering, Rajalakshmi Engineering College
balaji.s1.2024.cse@rajalakshmi.edu.in

[1]Department of Computer Science and Engineering, Rajalakshmi Engineering College

Faculty Guide: Mrs. B. Deepa, Assistant Professor (SG), Department of CSE
deepa.b@rajalakshmi.edu.in

ABSTRACT

Disasters create urgent information needs for affected people, responders, and coordinating organisations. During floods, cyclones, fires, earthquakes, industrial incidents, and other emergencies, a person may have limited time to explain what has happened, communicate an exact location, or identify the safest way to request assistance. Communication networks may be congested, infrastructure may be damaged, and people may be unfamiliar with official reporting channels. ResQAI is proposed as a citizen-oriented mobile application intended to make the first step of emergency communication more structured and accessible. The concept combines an SOS workflow, device-location support, incident reporting, optional media attachment, connectivity awareness, local notifications, and a service layer that can communicate with a backend when one is configured. The application is developed using Flutter and Dart, allowing a shared codebase and a component-based interface. Supporting packages in the project include navigation, state management, networking, geolocation, media selection, connectivity detection, and local preferences. These technologies provide building blocks, but their presence in a dependency file does not by itself prove that every feature is fully implemented or validated.

This paper describes the motivation, related work, proposed architecture, requirements, module design, implementation approach, preliminary evaluation plan, risks, and future improvements for ResQAI. It treats emergency communication as a socio-technical problem rather than merely a user-interface problem: clear interaction design must be supported by permission handling, reliable data transfer, privacy safeguards, backend processing, and operational partnerships. The paper distinguishes intended capabilities from verified results. No claim is made that the current prototype has delivered real emergency alerts to public agencies or that end-to-end performance has been established. The proposed evaluation therefore focuses on functional correctness, usability, location permission scenarios, network failure behaviour, data integrity, and safe handling of unsuccessful submissions. ResQAI is intended as a foundation for a more complete disaster-support system, subject to implementation, field testing, security review, and integration with authorised response organisations.

Keywords—disaster response, mobile application, SOS, geolocation, incident reporting, Flutter, citizen safety, emergency communication, resilience.

I. INTRODUCTION

A. Background

Disasters are not only physical events; they are also periods in which information becomes difficult to obtain, verify, and distribute. People need to know what is happening, where help may be available, which routes are unsafe, and how to communicate their needs. Response teams, in turn, need concise and actionable information to decide where to direct limited resources. When these two sides cannot exchange information effectively, delays and duplication may occur. The effect is especially serious when a person is injured, displaced, separated from family, or unable to provide a long verbal explanation.

Smartphones can support parts of this communication process because they combine a user interface, location sensors, a camera, connectivity, and notification mechanisms in one device. A carefully designed application can reduce the number of steps required to submit an incident report and can help a user include relevant details such as incident category, description, approximate coordinates, time, and optional media. However, a mobile application is not a replacement for emergency services, trained responders, public warning systems, or reliable communications infrastructure. Its value depends on clear boundaries and on whether the information reaches a responsible service.

B. Problem Statement

Citizens may encounter fragmented reporting methods during an emergency. A phone call may be difficult to complete; a message may omit location details; and a social-media post may not reach an authorised organisation. Some people may not know which information is most useful to provide. The problem addressed by this work is how to design a simple citizen-facing mobile workflow that gathers essential incident information, supports an SOS request, and communicates status transparently without overstating what the system can guarantee.

C. Aim and Objectives

The main aim is to design and progressively implement ResQAI as a mobile platform that organises the initial stages of citizen emergency reporting. The objectives are:

- Provide a prominent, understandable SOS interaction that reduces unnecessary navigation during a stressful situation.
- Support location permission checks and retrieval of device coordinates when available and authorised.
- Allow a citizen to submit a structured incident report with a category and descriptive details.

- Provide optional image or media selection where it is appropriate and permitted by the device.
- Represent network availability and submission outcomes so that users are not misled about delivery.
- Use modular application architecture so that navigation, state management, networking, and platform services can be maintained independently.
- Define a test plan for normal operation, denied permissions, unavailable location, offline use, and server errors.
- Identify the security, privacy, accessibility, and operational requirements that must be addressed before deployment.

D. Scope and Boundaries

The scope of this paper is a citizen mobile application prototype and its proposed interaction with a backend service. It covers the user-facing workflow and the software components needed to support it. It does not claim to provide a complete emergency dispatch centre, a verified geographic information system for all hazards, an authoritative early-warning feed, or direct integration with government emergency numbers. Such integrations require agreements, technical interfaces, security controls, service-level expectations, and testing with the responsible organisations. Any future deployment must clearly communicate the official channels available in each region.

E. Contribution

The contribution of this work is a structured design for a mobile-first citizen reporting workflow, a modular implementation outline, and a practical validation plan. Rather than reporting unsupported performance numbers, the paper identifies measurable evaluation criteria and explains how they can be assessed once a stable build and backend are available. This is important for safety-oriented software, where reliability claims must be supported by evidence.

II. LITERATURE REVIEW

A. Disaster Risk Reduction and People-Centred Systems

Disaster risk reduction emphasises understanding risk, preparing communities, communicating warnings, and coordinating action before and during hazardous events. The Sendai Framework for Disaster Risk Reduction 2015–2030 highlights the importance of understanding disaster risk, strengthening governance, investing in resilience, and improving preparedness for effective response. These priorities imply that information systems should be designed around the needs and capabilities of people who may be affected, rather than around technology alone. A citizen application can support one part of this broader system by improving the structure of information submitted by individuals.

People-centred communication also requires attention to inclusion. A system that depends on a recent smartphone, continuous data access, high literacy, precise location permission, or a single language may exclude people who need assistance. The application should therefore avoid unnecessary complexity, use plain language, provide readable controls, and explain alternatives when a device capability is unavailable. It should not assume that all users can safely type a long description or attach photographs.

B. Mobile Technologies in Emergency Contexts

Mobile devices are useful because they are familiar to many citizens and can collect several forms of information. A structured form can encourage consistent fields, while a location sensor may provide coordinates that are difficult to describe verbally. A camera can help document visible damage or a hazard when taking a photograph is safe. Notifications may provide local reminders or status updates. These capabilities are complementary: location can help identify where an incident occurred, text can explain what is happening, and media can provide additional context.

Nevertheless, mobile systems are constrained by battery life, radio coverage, operating-system restrictions, device diversity, and user permissions. Location accuracy can vary between indoor and outdoor environments and between precise and approximate permission modes. A device may return a position that is stale or unavailable. The interface should communicate uncertainty and allow the user to correct or supplement location details where feasible. A timestamp and the source of the location should be retained where the system design requires them.

C. Geolocation and Contextual Reporting

Geolocation packages expose operating-system services to application code. In Flutter, the Geolocator package supports obtaining current or last-known positions, checking location service status, and requesting location permissions. Developers must handle platform-specific permission requirements and the distinction between approximate and precise access [4]. A robust application should check whether location services are enabled, request permission at a suitable point in the workflow, handle denial without crashing, and avoid repeatedly prompting a user who has declined access.

Coordinates alone do not guarantee a useful report. A report may need incident category, brief description, approximate severity, time observed, and optional contact details. Each additional field can improve context but can also increase completion time. The design challenge is to collect the minimum information needed for triage while keeping the initial path short. Optional details should be clearly labelled so that a person in danger is not forced to complete a lengthy form before seeing available help options.

D. Cross-Platform Development and Maintainability

Flutter is a cross-platform UI toolkit that uses Dart and allows developers to share much of the application code across platforms. Its architecture separates widgets and framework services from the rendering engine and platform embedder [1], [2]. This layered structure is relevant to ResQAI because user-interface logic can be kept separate from platform-specific functions such as location access and media

selection. Shared code can simplify maintenance, but it does not eliminate the need to test on the target operating systems.

State management and routing libraries can help keep complex applications organised. Riverpod can represent application state and dependencies, while GoRouter provides declarative navigation patterns. Dio can encapsulate HTTP requests, timeouts, interceptors, and error handling. These tools are useful only when applied with clear boundaries: UI widgets should not contain every networking detail, and state changes should have defined success and failure outcomes. Intentional separation improves maintainability, testability, and scalability [1].

E. Communication Reliability and Offline Conditions

Connectivity detection can indicate whether a device appears to have a network interface, but it does not prove that a backend is reachable or that a request has been accepted. A device may be connected to Wi-Fi with no internet access, or the server may be unavailable. Therefore, the application should distinguish at least three states: no apparent connectivity, request in progress, and server-confirmed success or failure. The user interface should never label an SOS request as delivered merely because a button was pressed.

A later version could provide a locally saved draft or a queued report for retry, provided the user is informed that the report has not yet been sent. Queued emergency messages require careful design to prevent duplicates, stale information, and false reassurance. Each report should have a unique identifier, creation time, delivery status, and retry policy. A server should return an explicit acknowledgement that the client can display. These mechanisms are proposed design requirements and must be implemented and tested before being described as operational.

F. Gaps Identified

The literature and platform documentation suggest several practical design requirements: simple user interaction, permission-aware location capture, clear status feedback, inclusive design, separation of application layers, and verification of actual server delivery. A common risk in prototype descriptions is to confuse a package dependency or a mock screen with a working service. ResQAI addresses this documentation gap by separating design intent, implementation status, and evidence required for validation. Its next stage should measure whether users can complete reporting tasks and whether the system handles failure modes predictably.

III. PROPOSED SYSTEM

A. System Overview

ResQAI is proposed as a mobile application that guides a citizen from opening the app to initiating an SOS request or submitting an incident report. The client collects information that the user chooses to provide and, when permission is granted, may attach device coordinates. The client then validates the input, prepares a request, and communicates with a configured backend. The backend is responsible for validating the request, assigning an identifier, storing the report, and returning an acknowledgement. If a backend is not available or is not connected, the application must present that limitation clearly.

B. Conceptual Architecture

The system is divided into five logical layers. The presentation layer contains screens, forms, buttons, and feedback messages. The state and workflow layer manages the current report, validation status, loading indicators, and navigation. The device-services layer wraps location, media selection, connectivity status, local notifications, and local preferences. The network layer sends structured requests and handles timeouts, response codes, and parsing. The backend layer—when implemented and configured—validates requests, stores reports, records status changes, and provides authorised operational access.

```
CITIZEN
↓
PRESENTATION SCREENS
↓
WORKFLOW / STATE MANAGEMENT
↓
DEVICE SERVICES  ↔  LOCAL PREFERENCES
↓
NETWORK CLIENT (HTTP / API)
↓
BACKEND API → VALIDATION → REPORT STORE → AUTHORISED REVIEW
```

Figure 1. Conceptual architecture of ResQAI. This diagram represents the intended separation of responsibilities; it is not evidence that every layer is already connected or deployed.

C. Primary Workflow

A typical reporting workflow begins when the user selects SOS or incident reporting. The application displays concise guidance and obtains only the information required for the selected path. If location is requested, the application checks service availability and permission. If the user grants permission, coordinates can be retrieved and shown as part of the report. If permission is denied or location cannot be obtained, the workflow should continue where possible with a clear warning and a manual-location alternative if that feature is implemented. The user reviews the details and submits the report. The interface then displays a pending state until a backend response confirms the outcome.

A successful response should contain a report identifier and a meaningful status. A failure response should explain whether the user can retry, save a draft, or use another official channel. The application should avoid exposing internal exception text or technical stack traces. The workflow must also protect against accidental repeated taps, for example by disabling the submit action while a request is in progress and by using a unique client-generated request identifier if supported by the backend.

D. Functional Requirements

- FR-1: The application shall provide an accessible entry point for SOS-related actions.

- FR-2: The application shall allow a citizen to enter an incident category and description.
- FR-3: The application shall request location permission only when location functionality is needed.
- FR-4: The application shall handle denied, restricted, unavailable, and approximate location states.
- FR-5: The application shall allow optional media selection only where the feature is implemented and permissions allow it.
- FR-6: The application shall validate required fields before submission and explain missing information.
- FR-7: The application shall display a loading state and prevent accidental duplicate submissions.
- FR-8: The application shall distinguish local preparation from backend-confirmed receipt.
- FR-9: The application shall provide understandable failure feedback when connectivity or server requests fail.
- FR-10: The application should support a report history or status view if the backend provides the necessary data.

E. Non-Functional Requirements

Usability requires the most urgent actions to be discoverable, text to remain readable, and the form to avoid unnecessary cognitive load. Reliability requires explicit error handling for permissions, timeouts, malformed responses, and server errors. Security requires encrypted transport, server-side validation, controlled access to reports, and secure handling of credentials. Privacy requires collecting only necessary information, explaining the purpose of location and media, defining retention periods, and avoiding unnecessary background tracking. Maintainability requires modular services, consistent naming, and separation between UI and business logic. Accessibility requires sufficient contrast, scalable text, meaningful labels, and controls that do not rely on colour alone. Performance should be measured on representative devices and networks rather than assumed from the choice of framework.

F. Data Model Proposal

A proposed incident record may include report_id, created_at, incident_type, description, latitude, longitude, location_accuracy, location_permission_state, optional_media_reference, client_status, server_status, and an optional contact method. Not every field needs to be collected for every report. Sensitive values should be minimised, and any user identifier should be generated or managed according to the application's authentication design. The backend should validate field types, coordinate ranges, allowed incident categories, file size, and media type. The client must not be treated as a trusted source because requests can be modified outside the app.

G. Security and Privacy Considerations

Location and incident details can reveal a person's movements, vulnerabilities, or circumstances. Access should be limited to roles with a legitimate operational need. The system should use HTTPS, authentication for privileged interfaces, server-side authorisation, secure secrets management, input validation, and audit logs for access to sensitive reports. Media should be scanned or safely processed before it is made available to other users. Logs should avoid recording full sensitive descriptions or precise coordinates unless operationally necessary. A privacy notice should explain what is collected, why it is needed, who may receive it, and how long it is retained.

The app should also consider misuse. False reports, repeated submissions, malicious media, and attempts to impersonate another person may burden responders. Rate limiting, abuse monitoring, duplicate detection, and review procedures can reduce risk, but must not prevent genuine users from requesting help. Any automated classification or prioritisation introduced in the future should be explainable, evaluated for bias, and subject to human oversight.

IV. MODULES DESCRIPTION

A. Module 1: SOS Workflow

The SOS workflow is the primary interaction path for a citizen who believes that immediate assistance is needed. The interface should make the action prominent but also avoid accidental activation. A short confirmation step may be appropriate for some actions, but excessive confirmation can delay a user who is already in danger. The exact design should be tested with users and should not imply that pressing SOS automatically contacts police, ambulance, fire services, or disaster-management authorities unless a verified integration exists.

At a minimum, the workflow should manage initial state, user action, validation, request preparation, submission, acknowledgement, and error states. While a request is being sent, the interface should prevent repeated submissions and communicate that processing is in progress. If the request succeeds, it should show the report identifier and the response returned by the service. If it fails, the app should clearly state that delivery could not be confirmed and direct the user to appropriate official channels. A failure should never be silently converted into a success state.

The SOS module should be tested against rapid repeated taps, navigation away during submission, permission denial, no network, slow network, server timeout, invalid response, and application restart. If the system later supports retry, the retry mechanism must avoid creating duplicate reports. A report may need a stable idempotency key so the backend can recognise repeated attempts to submit the same request.

B. Module 2: Location Acquisition

The location module provides a controlled interface to device geolocation services. Before obtaining coordinates, the application checks whether location services are enabled and whether the user has granted permission. When appropriate, it requests permission and then attempts to retrieve a current position. The module should distinguish a fresh position from a last-known position, and should record accuracy metadata when available. If no position can be obtained within a

reasonable timeout, the user should receive a clear explanation rather than an indefinite loading indicator.

Location access is sensitive and must be contextual. The application should not request continuous background tracking unless there is a clearly justified requirement and the user has been informed. The user should be able to understand why location is being requested and how it contributes to the report. Approximate location may be sufficient for some use cases; precise location may be needed for others. The app must handle the operating system's actual permission result rather than assume that the user granted the level requested.

The proposed location payload includes latitude, longitude, capture time, and accuracy where available. The server should validate that coordinates are numeric and within valid geographic ranges. If the user edits the location or enters a landmark, the system should distinguish user-entered information from sensor-derived information. This distinction helps reviewers assess uncertainty and avoids presenting an approximate coordinate as a verified address.

C. Module 3: Incident Reporting

Incident reporting provides a structured alternative to a free-form message. A category list can help group incidents such as flooding, fire, road blockage, structural damage, medical assistance, or other hazards. The category set should be configurable and reviewed with domain experts. A description field permits details that do not fit into predefined categories. Required fields should be kept to a minimum; optional fields can collect additional context when the user is able to provide it safely.

Validation should be understandable and immediate. For example, if a description is required, the app should identify the field and explain what is missing. Text length limits should be chosen to preserve useful context while preventing excessive payloads. The application should not ask for unnecessary identity data. Where contact details are needed, the purpose should be explained, and secure handling should be designed at the backend.

D. Module 4: Media Attachment

Photographs can clarify visible damage or hazards, but collecting media during an emergency may distract or endanger a person. Media attachment should therefore be optional and should never be framed as a condition for asking for help. The module should handle cancellation, unsupported file types, large files, permission denial, and unavailable camera or gallery services. Images should be compressed when appropriate, while retaining enough detail for the intended use.

A complete production implementation should consider file metadata, embedded location information, malware scanning, content moderation, upload progress, and secure storage. Access to uploaded media should be limited to authorised reviewers. The interface should tell users whether the image is still on the device, currently uploading, or successfully received by the service. These statuses are distinct and should not be conflated.

E. Module 5: Connectivity and Network Communication

The network module should centralise API calls and response parsing. Dio can be configured with a base URL, connection and receive timeouts, interceptors, and common error handling. The base URL should be configured for each environment instead of being scattered throughout screen code. Development, testing, and production environments must be kept distinct to avoid sending test reports to a live service.

Connectivity information is only a preliminary signal. Before declaring a report delivered, the app needs a positive acknowledgement from the backend. The client should handle HTTP errors, DNS failures, timeouts, invalid JSON, authentication failures, and server-side validation messages. Retry policies should be conservative for non-idempotent operations. If the backend is not configured, the app should show that submission is unavailable rather than imply a live emergency dispatch.

F. Module 6: Navigation and State Management

Navigation defines how a user moves among home, SOS, incident form, location preview, confirmation, and status screens. A predictable route structure reduces confusion and makes it easier to test whether the back button returns the user to a safe state. GoRouter can define named routes and navigation rules. The application should avoid losing an unsent report without warning when the user navigates away.

State management can represent the report lifecycle: draft, validating, submitting, accepted, failed, or saved locally. Riverpod can provide testable state and dependency boundaries when used consistently. A state transition model is particularly helpful for preventing contradictory UI messages, such as displaying both “sending” and “sent.” It also allows tests to simulate location failures and network responses without requiring every test to interact with physical hardware.

G. Module 7: Local Preferences and Notifications

Local preferences may store non-sensitive choices such as language or display preferences. Sensitive incident content and authentication secrets should not be placed in ordinary preferences without an appropriate security design. Local notifications can remind users about an application state or provide a local message, but they are not equivalent to a server-confirmed emergency alert. If a future version sends remote status updates, it will require a server-side notification service and explicit handling of notification permissions.

H. Module 8: Accessibility and User Guidance

The interface should use plain language, short instructions, large touch targets, and clear visual hierarchy. Important status messages should be readable by screen readers and should not rely solely on colour. Forms should preserve entered text after recoverable errors. Multilingual support may be important in regions where users speak different

languages, and the wording should be reviewed by fluent speakers rather than translated mechanically. For users with limited literacy, icon labels and short guided steps may help, but icons should not replace text entirely.

A safety-oriented design should also offer a clear route to official emergency contact information appropriate to the deployment region. Such information must be maintained and verified. The app should not give unqualified medical or evacuation instructions unless these have been reviewed by relevant authorities. When the application cannot transmit a report, it should direct the user toward available alternatives without suggesting that help has already been dispatched.

V. IMPLEMENTATION METHODOLOGY

A. Development Environment

The project is organised as a Flutter application written in Dart. The current project dependency set includes packages associated with state management, navigation, HTTP communication, maps and coordinates, geolocation, permission handling, media selection, connectivity, URL launching, date/time formatting, local preferences, notifications, and speech-to-text. These dependencies indicate the intended technical direction; each capability still requires code-level integration and verification. The app should be built and tested on the target platform using a supported Flutter SDK and a configured Android emulator or physical device.

A repeatable development process begins with dependency resolution, static analysis, and a successful debug build. The next stage verifies each screen in isolation, followed by module tests and end-to-end workflow tests. API-dependent tests should use a controlled test server with predictable responses. Configuration values, such as the API base URL, should be supplied per environment. On an Android emulator, a host machine's local backend may require the emulator-specific host address; a browser target generally uses a different localhost configuration.

B. Suggested Layer Separation

The presentation layer should contain widgets and screen-specific interaction. Providers or controllers should manage application state and expose simple operations such as requestLocation, validateReport, submitReport, and refreshStatus. Device adapters should encapsulate platform calls so that UI tests can replace them with fakes. A repository should provide a stable interface for report creation and status retrieval. The API client should be responsible for serialization, timeouts, response parsing, and transport errors. This separation reduces coupling and allows the same workflow to be tested without requiring a real GPS signal or a live emergency backend.

C. Error Handling Strategy

Errors should be classified by user impact rather than displayed as raw technical exceptions. Examples include permission denied, location unavailable, offline device, request timeout, server rejection, expired authentication, and unexpected server response. Each category should map to a safe user message and a recovery action. Developers should retain diagnostic logs in development builds while excluding sensitive report content from logs. In production, telemetry must follow the privacy policy and avoid capturing personal data unnecessarily.

D. Development and Verification Stages

- Stage 1 — Requirements: define the emergency scenarios, target users, and boundaries of the prototype.
- Stage 2 — Interface: build navigation and forms with clear empty, loading, success, and failure states.
- Stage 3 — Device services: implement and verify location, permission, media, connectivity, and local settings.
- Stage 4 — Backend contract: define request/response schemas, authentication, validation, storage, and acknowledgement semantics.
- Stage 5 — Integration: connect the UI to the service layer and verify state transitions.
- Stage 6 — Testing: run unit, widget, integration, accessibility, security, and failure-mode tests.
- Stage 7 — Field readiness: conduct supervised user evaluation and review the workflow with authorised response stakeholders.

E. Implementation Status and Evidence

At the time of writing, this paper documents the project design and implementation direction. A complete successful build, end-to-end test, and live emergency-service integration have not been established by the evidence available for this report. Accordingly, the following sections describe a proposed experimental setup and evaluation plan, not a claim that the application has passed every test. Before submission, the author should update this section with the actual build date, device or emulator, operating-system version, test outcomes, and screenshots obtained from the running application.

VI. EXPERIMENTAL SETUP AND EVALUATION PLAN

A. Test Environment

The minimum test environment should include a development computer, a supported Flutter SDK, an Android emulator or physical Android device, a controlled API endpoint, and test data that contains no real personal information. A second device or browser can be useful for reviewing responsive behaviour where supported. The test log should record the application build identifier, device model, operating-system version, network condition, permission state, test input, expected result, observed result, and whether the test passed.

Testing should begin in a non-production environment. The backend should provide deterministic responses for accepted reports, validation errors, unauthorised requests, server failures, and slow responses. Simulated coordinates should be used for repeatable tests, while real location testing should be

conducted only with informed participants and appropriate privacy safeguards. No real emergency call should be triggered during a software test.

B. Functional Test Cases

Scenario	Expected behaviour
Open application	Home screen loads without crash
Empty required fields	Validation explains missing fields
Grant location permission	Location result is handled
Deny location permission	App remains usable; limitation shown
Location services disabled	Clear recovery message appears
Select then cancel media	Form remains stable
No network	Not reported as delivered
Backend accepts request	Acknowledgement and report ID shown
Backend validation error	Safe error message displayed
Backend timeout	Timeout and retry guidance appear
Repeated submit taps	Duplicate prevented or deduplicated
Navigate away with draft	Data-loss behaviour is predictable
Invalid/large media	Attachment safely rejected
Screen reader/large text	Controls remain operable
Application restart	No false success state appears

Table 1. Proposed functional test cases. These are planned checks; results must be filled in after execution.

C. Metrics

Evaluation should include task completion rate, time to complete a report, number of validation errors, location acquisition success rate, percentage of submissions acknowledged by the backend, median and high-percentile request latency, duplicate submission rate, crash rate, and user-reported clarity. Metrics must be defined before testing. For example, a submission acknowledgement rate should use only attempts made under a defined network and server condition; it should not combine offline attempts with server failures without explanation.

Usability testing should ask participants to complete realistic but non-emergency tasks, such as reporting a simulated road blockage or flooding event. Participants should be told that the system is a prototype and does not contact emergency responders. Researchers should observe where participants hesitate, misunderstand status messages, or provide incorrect information. A small pilot can identify obvious usability problems, but a larger and more diverse sample is needed before making broad claims about usability.

D. Results Reporting Template

Because verified test outputs were not available when this paper was prepared, numerical performance results are intentionally not fabricated. After testing, the results section should state the number of test runs, device and software versions, network conditions, failures encountered, and corrective actions. Each result should be traceable to a log, test report, or reproducible observation. Screenshots should come from the actual application build and should be captioned with the screen name and the scenario represented.

E. Threats to Validity

Prototype evaluation can be affected by limited device coverage, small participant samples, simulated network conditions, and a backend that does not represent a production service. A successful test on one emulator cannot establish reliability across all phones or networks. Location accuracy measured in one environment may not generalise to high-rise buildings, rural areas, dense urban streets, or indoor settings. User feedback may also be influenced by familiarity with smartphones. These limitations should be disclosed rather than hidden.

F. Safety-Centred Acceptance Criteria

Before the application is considered ready for public use, the project should demonstrate that it never claims delivery without acknowledgement, handles permission denial gracefully, protects sensitive data, prevents avoidable duplicate submissions, and provides an alternative when network transmission fails. Security review should cover authentication, authorisation, transport encryption, media handling, and log retention. Operational review should confirm who receives reports, who is responsible for responding, what service hours apply, and how false or abusive reports are handled. These criteria are as important as visual polish.

VII. DISCUSSION

A. Expected Benefits

If implemented and integrated correctly, ResQAI may help citizens submit more consistent incident information and reduce avoidable back-and-forth caused by missing details. A common form can make reports easier to review, while optional coordinates may help responders locate an incident. A clear status model can reduce uncertainty for the user and support more trustworthy communication. The modular design also gives developers a path to add features incrementally rather than building one large, tightly coupled screen.

B. Operational Dependencies

The quality of the mobile client is only one factor in emergency response. The receiving organisation needs staff, procedures, geographic coverage, prioritisation rules, and a way to act on incoming information. If the service is not monitored, a report may be stored without prompting a response. If the backend is unavailable, the client may be unable to transmit data. Therefore, ResQAI should be presented as a communication aid and not as a guarantee of rescue or dispatch. Partnerships and operational agreements are prerequisites for any real deployment.

C. Ethical and Inclusion Considerations

The application should be designed for people with different ages, languages, disabilities, device capabilities, and levels of digital confidence. Location data should be used only for the stated purpose and should not be repurposed for unrelated tracking. People should not be pressured to share media or unnecessary identity information. Where the system is used by vulnerable populations, privacy and safety risks may be greater, and a formal impact assessment may be appropriate. Automated prioritisation should not be introduced without evidence, transparent criteria, and human oversight.

D. Limits of the Current Prototype

The current project should be treated as a prototype until its build, workflow, backend, and test evidence are complete. The presence of Flutter packages does not prove that all

associated modules work together. A visible SOS button does not establish that a real emergency service receives the request. A map or location permission does not establish that the coordinates are accurate enough for dispatch. A local notification does not establish that an external responder has been notified. These distinctions guide the next development phase and keep the paper's claims aligned with available evidence.

VIII. USE-CASE ANALYSIS

A. Individual Reporting a Local Hazard

Consider a citizen who notices a flooded road, a fallen electrical line, or damage to a building. The user opens the application, chooses the most suitable incident category, and provides a short description. If location permission is granted, the application attempts to obtain coordinates and may include accuracy information. The user should be able to review the report before sending it. A clear summary reduces the likelihood that the citizen submits the wrong category or an incorrect location. After submission, the application should wait for the backend response and show a report reference only when one has actually been returned. This workflow is useful for non-immediate hazards as well as for documenting conditions that may require assessment.

B. User Requiring Urgent Assistance

An urgent request has different interaction needs from a routine incident report. The path should be short, labels should be direct, and optional fields should not block the user from continuing. If the prototype does not connect to an authorised emergency service, the interface must state this clearly and provide region-appropriate official contact options where verified. The design should not use reassuring phrases such as “help is on the way” unless a responsible service has explicitly confirmed that status. If a request fails, the user should be told that delivery could not be confirmed. This distinction is central to safety and user trust.

C. Citizen with Location Permission Denied

A user may deny location permission because of privacy concerns, device settings, or a preference to enter a location manually. The application should not repeatedly interrupt the user with permission prompts. It should explain why location could help and continue with any safe alternative that the product supports, such as a landmark or text description. If a manual-location field is not implemented, the limitation should be explained rather than silently submitting an empty coordinate. This scenario should be included in usability tests because a system that works only when every permission is granted is fragile in real-world conditions.

D. User on an Unreliable Network

A citizen may have intermittent cellular coverage or Wi-Fi access without a functioning internet connection. The client can check connectivity as an initial clue, but the definitive outcome depends on the network request and server acknowledgement. A request that times out may have reached the server even if the response was lost, so blindly retrying can create duplicate reports. A production backend should support idempotency or duplicate detection, and the user interface should explain the uncertainty. A locally saved draft may help preserve the entered details, but the app must label it as unsent until server receipt is confirmed.

IX. COMPARATIVE DESIGN ANALYSIS

A. Unstructured Communication Versus Structured Reporting

A phone call or informal message allows flexibility, but the details provided can vary widely. One person may describe the hazard clearly while another may omit location, incident type, or time. A structured reporting form encourages consistency and may make later sorting easier. However, a form can also introduce friction if it demands too many fields. ResQAI therefore adopts a minimal-required-information principle: collect essential details first, and make additional context optional where possible. The design should be evaluated to confirm that structure improves information quality without making urgent reporting slower.

B. Manual Location Versus Device Location

Manual location entry can be useful when a device location is unavailable or inaccurate, but users may not know street names or coordinates. Device location can reduce the effort of describing a position, but it depends on permissions, sensor availability, environmental conditions, and accuracy. A resilient design treats these methods as complementary. Where implemented, the application can display the retrieved position and allow the user to add a landmark or correct the location. The system should retain enough metadata to distinguish sensor-derived coordinates from user-provided descriptions.

C. Local Feedback Versus Confirmed Delivery

A button animation, local notification, or change of screen can confirm that the application reacted to an interaction, but none of these events proves that an external service received the request. A server acknowledgement confirms only that the configured backend accepted the request according to its own contract; it does not prove that a responder has read or acted on it. If the product later introduces review statuses, each status must be tied to an explicit event from the responsible service. Separating these levels of confirmation avoids false confidence and makes the system easier to audit.

X. RISK ANALYSIS AND MITIGATION

A. Technical Risks

Technical risks include application crashes, incorrect state transitions, API outages, invalid responses, stale coordinates, media upload failures, and differences among Android versions or device manufacturers. Mitigation begins with a reproducible build process, automated tests, clear error handling, and monitoring of non-sensitive operational metrics. The project should use controlled test environments and versioned API contracts. Dependencies should be reviewed and updated deliberately rather than changed

immediately before a demonstration without regression testing.

B. Safety and Human-Factor Risks

Safety risks include a user misunderstanding whether an alert was delivered, taking a photograph in a dangerous location, spending too long completing a form, or relying on the app instead of an official emergency channel. The interface should minimise unnecessary steps, avoid coercing users into media capture, and communicate limitations in plain language. User testing should include realistic time pressure without exposing participants to actual danger. Safety messages and official contact details should be reviewed by qualified stakeholders and kept current.

C. Privacy and Security Risks

Incident reports can contain precise locations, personal narratives, images, and contact details. A breach could expose vulnerable people or reveal sensitive information about a disaster area. Data minimisation, encrypted transport, access control, server-side validation, secure storage, retention limits, and audit logging should be treated as core requirements. Test datasets should use fictional information. The development team should avoid placing credentials in source code or including sensitive details in crash logs. Before deployment, the system should undergo a threat model and security review appropriate to its intended use.

D. Operational Risks

Even a technically correct application may fail to produce a response if nobody is assigned to receive and triage reports. Operational risks include unclear ownership, unmonitored queues, inconsistent categories, duplicate reports, and lack of agreed response procedures. The receiving organisation must define responsibility for each report type, expected operating hours, escalation rules, and closure criteria. If no organisation has accepted responsibility for monitoring reports, the app must not imply that it provides a staffed emergency response service.

XI. DATA FLOW, VALIDATION, AND REPORT LIFECYCLE

A. Report Creation and Validation

When a citizen starts a report, the client should create a draft state rather than immediately sending partial data. Field validation should check required values, allowed category values, description length, media constraints, and location format. Validation on the client improves usability but is not a security boundary. The backend must repeat relevant checks because clients can be modified or bypassed. A well-defined request schema makes this contract testable and reduces differences between client and server expectations.

B. Submission and Acknowledgement

After local validation, the client submits a request over an encrypted connection. The backend should authenticate the request when required, validate the payload, apply rate limits, persist the report, and return a stable report identifier with a defined status. The client should display the returned identifier only after receiving and parsing a valid acknowledgement. If the response is malformed or the connection drops, the client should not assume success. Where the server may have processed a request despite a timeout, an idempotency key or a status lookup endpoint can help reconcile the outcome.

C. Status Transitions

A suggested lifecycle is Draft, Ready to Submit, Submitting, Accepted by Server, Under Review, Resolved, or Failed. Not every state needs to be exposed to the citizen, and the exact labels depend on the backend contract. The client should not invent statuses based on elapsed time or screen navigation. State transitions should be explicit and validated, so that a report cannot move from Draft directly to Resolved without a trustworthy event. Audit information should record relevant changes while avoiding unnecessary personal data.

D. Data Retention and Deletion

A production system should define how long report data, media, logs, and backups are retained. Some information may be needed for operational follow-up or legal obligations, but indefinite retention should not be assumed. Deletion and correction requests require a documented policy and clear ownership. If reports are shared with multiple agencies, the privacy notice should explain those recipients and the limits of deletion after onward transfer. Retention decisions should be reviewed with legal, security, and operational stakeholders before deployment.

XII. FUTURE TECHNICAL ENHANCEMENTS

A. Multilingual and Accessible Interfaces

A future version can support multiple languages, scalable text, screen-reader semantics, high-contrast options, and simplified workflows. Translation should be reviewed by fluent speakers and, where emergency terminology is involved, by relevant domain experts. Accessibility should be tested with actual assistive technologies rather than inferred from visual inspection alone. These changes can broaden access, but should not be added without ensuring that critical instructions remain consistent across languages and display modes.

B. Responder Dashboard and Geographic Visualisation

An authorised dashboard could display incoming reports, filters, timestamps, locations, and status history. Geographic visualisation may help reviewers recognise clusters or identify incidents near vulnerable infrastructure. Such a dashboard introduces additional requirements for authentication, role-based permissions, audit trails, secure map services, and protection against exposing precise citizen locations to unauthorised users. It should support human decision-making rather than automatically determine the severity of an incident without evaluation.

C. Verified Alerts and External Integrations

Future integration with official warning feeds or emergency organisations could make the system more useful, but only

after the source, update frequency, geographic coverage, and meaning of each alert are verified. External APIs may be unavailable or may change. The application should identify the source and timestamp of a warning and avoid presenting outdated information as current. Any direct dispatch integration needs formal approval, service monitoring, privacy agreements, and end-to-end tests with the receiving organisation.

D. Responsible Use of Artificial Intelligence

Although the project name includes "AI," automated classification should not be assumed to exist unless it is implemented and validated. A future model might help group similar reports, identify incomplete submissions, or suggest categories. It should not make unreviewed life-critical decisions or replace trained responders. Evaluation would need representative data, performance measurement across incident types and languages, bias analysis, privacy review, explainability, and human oversight. A deterministic rules-based workflow may be more appropriate than machine learning for some early versions.

XIII. CONCLUSION AND FUTURE WORK

This paper presented ResQAI, a proposed intelligent disaster response citizen application focused on SOS initiation, location-aware incident reporting, optional media, network communication, and transparent feedback. It described the motivation for a structured mobile reporting workflow, reviewed relevant disaster-risk and mobile-development concepts, and outlined a modular architecture using Flutter and Dart. The design separates presentation, state management, device services, networking, and backend responsibilities to improve maintainability and testability.

The paper also identified safety requirements that extend beyond the user interface. A trustworthy system must handle denied permissions, unavailable location, weak connectivity, server errors, duplicate submissions, and sensitive data carefully. Most importantly, it must distinguish a request prepared on the device from a report acknowledged by a backend and from an emergency service actually acting on that report. No measured performance or live dispatch result is claimed in this version because a verified end-to-end evaluation was not available.

Future work should complete and validate the application build, connect a controlled backend, define a stable API contract, implement secure storage and access control, and conduct functional and usability testing. Additional work may include multilingual interfaces, accessible interaction patterns, map-based context, offline draft handling, verified public warning feeds, responder dashboards, and integrations approved by local authorities. Any use of artificial intelligence for incident classification or prioritisation should follow a separate evaluation, bias review, and human-in-the-loop policy. Field testing with relevant stakeholders is necessary before the application can be considered for real emergency use.

REFERENCES

[1] Flutter Team, "Architecting Flutter apps," Flutter documentation. Available: https://docs.flutter.dev/app-architecture. Accessed: Oct. 22, 2026.

[2] Flutter Team, "Flutter architectural overview," Flutter documentation. Available: https://docs.flutter.dev/resources/architectural-overview. Accessed: Oct. 22, 2026.

[3] United Nations Office for Disaster Risk Reduction (UNDRR), "Sendai Framework for Disaster Risk Reduction 2015–2030," United Nations, 2015. Available: https://www.undrr.org/publication/sendai-framework-disaster-risk-reduction-2015-2030.

[4] Geolocator contributors, "geolocator: Flutter geolocation plugin," pub.dev. Available: https://pub.dev/packages/geolocator. Accessed: Oct. 22, 2026.

[5] Dart Team, "Dart documentation," Dart. Available: https://dart.dev/guides. Accessed: Oct. 22, 2026.

[6] Riverpod contributors, "Riverpod: A reactive caching and data-binding framework." Available: https://riverpod.dev/. Accessed: Oct. 22, 2026.

[7] Flutter Community, "go_router package," pub.dev. Available: https://pub.dev/packages/go_router. Accessed: Oct. 22, 2026.

[8] Flutter Community, "Dio: A powerful HTTP client for Dart," pub.dev. Available: https://pub.dev/packages/dio. Accessed: Oct. 22, 2026.

[9] World Health Organization, "Health emergency and disaster risk management framework," WHO, 2019. Available: https://www.who.int/publications/i/item/9789241516181.

[1]Department of Computer Science and Engineering, Rajalakshmi Engineering College
saisanjay.ss.2024.cse@rajalakshmi.edu.in

[2]Department of Computer Science and Engineering, Rajalakshmi Engineering College
balaji.s1.2024.cse@rajalakshmi.edu.in

Faculty Guide: Mrs. B. Deepa, Assistant Professor (SG), Department of CSE
deepa.b@rajalakshmi.edu.in